CodeMafia: Technical Specification and Development Roadmap

1. Executive Summary & Strategic Positioning

CodeMafia is a high-stakes technical platform engineered as a Python-based, secure, real-time adversarial debugging environment. Moving beyond the trivialities of social-deduction games, CodeMafia serves as a rigorous showcase for expertise in distributed systems, network synchronization, and cybersecurity. The interaction model tasks 3–5 players with collaboratively fixing broken Python algorithms in a shared Monaco environment, while one "Impostor" performs subtle sabotage.

As a capstone project, CodeMafia is designed to demonstrate advanced proficiency in Data Structures and Algorithms (DSA) and systems architecture. Unlike standard CRUD applications, this platform integrates an "Ability Manager" where Impostors unlock sabotage tools—such as "Network Blackouts" or "Process Freezes"—by correctly answering high-level Computer Science questions spanning Operating Systems, Networks, and Security. This mechanism transforms the game into a verified demonstration of domain knowledge. To ensure platform integrity, the system utilizes browser-side WebAssembly (Wasm) for execution and a rigid server-authoritative state machine to manage the adversarial logic.

2. System Architecture & Authoritative Policies

The CodeMafia architecture is strictly "Server-Authoritative." In an adversarial multiplayer context, any client-side authority constitutes a security vulnerability. This model prevents information leakage—ensuring the Impostor's identity remains hidden in the network traffic—and guarantees that the server remains the sole arbiter of the game's "Source of Truth."

The Golden Rule: Client Renders and Executes; Server Decides

To maintain a secure environment, the platform adheres to a binary responsibility model: the client handles UI feedback and local execution for the user, but the server owns the canonical state of the code and the final validation of win conditions.

Feature	Client-Side Responsibilities	Server-Side Responsibilities
Code Authority	Suggestions via debounced events	**Canonical Code String** (Master)

Execution	Local UI feedback via Pyodide (Wasm)	Hidden test validation & win conditions
Identity & Roles	UI rendering of player list	**Private role delivery** (No leakage)
Game Logic	Animations & visual "glitch" effects	State Machine, phases, & vote tallying
Synchronization	Monaco input & local state updates	Event broadcasting & conflict resolution

Architecture Shift Analysis: From Peer-to-Peer to Server-Authoritative

The project has undergone a strategic pivot from a Yjs-based client-authoritative model to a FastAPI WebSocket architecture. While Yjs provides excellent collaborative features via CRDTs, its peer-to-peer nature is fundamentally unsuitable for adversarial play. In Yjs, any client can theoretically broadcast a state change that overwrites others without server mediation. The revised FastAPI architecture implements **per-event validation**, ensuring the server verifies the player's status (is_alive) and current game phase before accepting any code updates. This shift establishes a zero-RCE (Remote Code Execution) environment on the backend while providing the granular authorization necessary for competitive integrity.

3. The Aligned Technical Stack

The selected stack balances low-latency performance with a "Python-first" requirement, ensuring the project serves as a relevant portfolio piece for backend and systems roles.

- **Core Language: Python (Backend & Engine)**. Used for the game logic, challenge database, and validation layer.
- **Backend Framework: FastAPI**. Configured as a modular monolith to handle asynchronous HTTP APIs and persistent WebSocket connections with minimal overhead.
- **Real-Time Layer: WebSockets**. Replaces heavy CRDTs with a server-broadcast model. Utilizing 100ms client-side debouncing, the server maintains the canonical code string and broadcasts atomic updates to all clients.
- **Frontend Foundation: HTML5, CSS3, and Vanilla ES Modules**. Avoids framework bloat to prioritize the performance of the editor and real-time event handling.
- **Code Environment: Monaco Editor + Pyodide (Wasm)**. Monaco provides the professional IDE interface, while Pyodide executes Python in a dedicated Web Worker, isolating the execution runtime from the main UI thread.

Data & Validation Layer

The persistence layer utilizes **SQLite** with **SQLAlchemy ORM** for managing the challenge library and game history. Critically, **Pydantic** is employed to enforce a **Strict Schema** for all JSON payloads. This ensures that every WebSocket event is validated against a predefined model, preventing malformed input attacks and stabilizing the state machine.

Security & Isolation: The "Untrusted Client" Reality

CodeMafia's security posture assumes the browser is untrusted. By running Pyodide in a WebAssembly sandbox, the system achieves a "Zero-RCE" state on the server. However, because Pyodide runs on the client, the server cannot blindly trust a "Tests Passed" message. The platform addresses this by maintaining **hidden test parameters** known only to the server. While Pyodide provides immediate feedback to the player, the final win/loss determination is calculated via server-side logic, demonstrating a cybersecurity-centric approach to system design.

4. The Revised 7-Phase Development Roadmap

The roadmap is structured to prioritize the core editor engine and security foundation before layering on multiplayer complexity and game mechanics.

1. **Phase 1: Local Editor Sandbox**
    - **Technical Objective:** Integration of Monaco with an isolated Pyodide Web Worker.
    - **Success Milestone:** Secure local Python execution where infinite loops do not freeze the UI thread.
2. **Phase 2: FastAPI WebSocket Infrastructure**
    - **Technical Objective:** Implementation of room partitioning, host re-election logic, and unique room code generation.
    - **Success Milestone:** Multi-client synchronization of player presence with no state leakage between rooms.
3. **Phase 3: Debounced Synchronisation & Atomic Updates**
    - **Technical Objective:** Replacing CRDT logic with a 100ms debounced broadcast model for the canonical code string.
    - **Success Milestone:** Resolution of race conditions; two players can edit the same block without "cursor jumping" or state desync.
4. **Phase 4: State Machine & Private Role Delivery**
    - **Technical Objective:** Implementing the core loop (LOBBY -> PLAYING -> MEETING) with zero-leakage role assignment.
    - **Success Milestone:** Secure delivery of the Impostor role to a specific socket ID, verified absent from global `gameState` broadcasts.
5. **Phase 5: Interruptive Meetings & Authoritative Tallying**
    - **Technical Objective:** Global editor locking mechanisms and server-side vote validation.
    - **Success Milestone:** Server successfully halts all code updates during the MEETING phase and resolves eliminations based on atomic vote counts.
6. **Phase 6: Challenge Database & Ability Manager**

    - **Technical Objective:** Integrating SQLAlchemy for the CS question/challenge database and linking it to the Impostor's ability cooldowns.
    - **Success Milestone:** Impostor successfully unlocks a "glitch" effect by passing a server-validated networking or OS quiz.
7. **Phase 7: Cybersecurity Hardening & Performance Polish**
    - **Technical Objective:** Implementing Rate Limiting, WebSocket Heartbeats, and payload schema enforcement.
    - **Success Milestone:** Platform stability under high-frequency event simulation and proof of resistance against malformed input attacks.

Dependency Analysis

The development sequence is strictly ordered to ensure the server-authoritative logic (Phase 4) is established before collaborative voting (Phase 5). This ensures that every player action—from typing a line of code to casting a vote—is validated against the player's life status and the current game phase, upholding the platform's commitment to professional-grade system integrity.